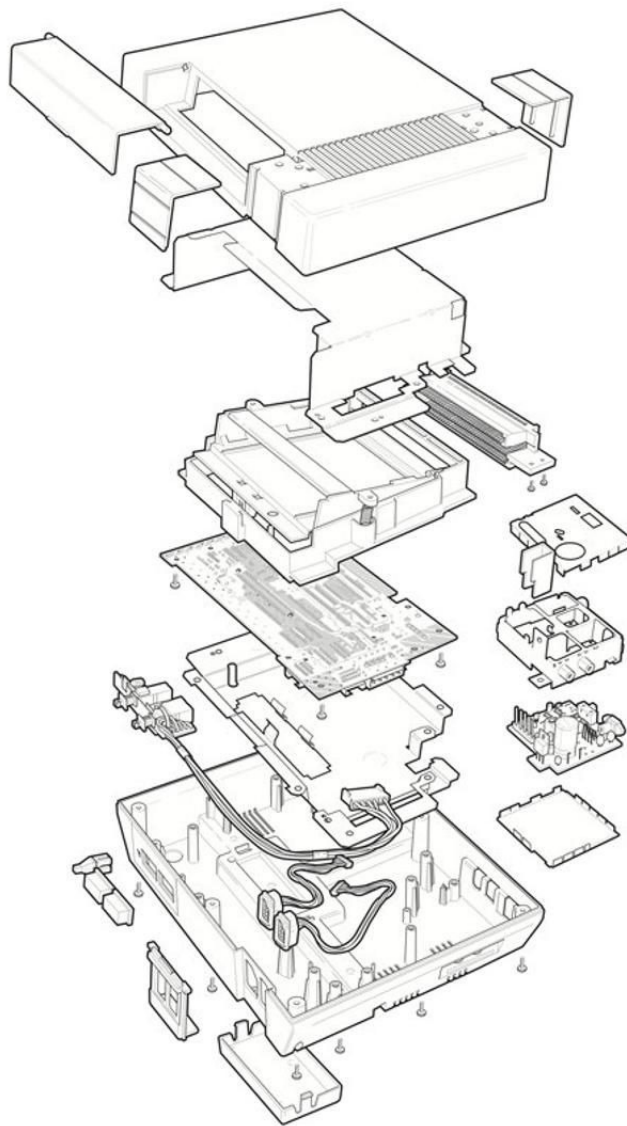


Elaborato di sicurezza del software: La sicurezza nelle console di gioco



Indice

1	Introduzione	3
2	Vulnerabilità Software: Memory Corruption	4
3	Protezione della copia e crittografia White-Box	6
4	Attacchi Hardware: Fault Injection e Glitching	8
5	Architetture Moderne	10
6	Conclusioni e considerazioni	11
7	Fonti	12

1 Introduzione

Lo scopo di questo elaborato è quello di analizzare come le tecniche offensive e difensive nelle console di gioco casalinghe si siano evolute nel tempo.

Il modello di minaccia che caratterizza questi sistemi è il modello MATE – *Man At The End*, ovvero quello in cui l'attaccante ha completa disposizione fisica della macchina.

Analizziamo quindi lo stato dell'arte delle tecniche di difesa impiegati dalle più grandi case produttrici di console di gioco, e di come siano comunque state prese di mira attraverso *exploit* software, corruzione della memoria e attacchi hardware come la *Fault Injection*, con l'obiettivo di aggirare la *Chain of Trust*.

1.1 Dispositivi Chiusi e l'Ecosistema *Walled Garden*

Il mercato dell'elettronica di consumo si divide storicamente in piattaforme aperte (PC Windows/Linux) e piattaforme chiuse (console, *Set-Top Box*, dispositivi iOS). Una tendenza consolidata mostra che l'apertura delle piattaforme attira un numero maggiore di sviluppatori e consumatori, ma abbassa drasticamente la soglia di competenza richiesta a un attaccante per comprometterle.

Per mantenere il controllo e proteggere la proprietà intellettuale, il cui valore commerciale risiede principalmente nel software e non nell'hardware, ci produttori adottano il paradigma del Dispositivo Chiuso, creando un ecosistema iper-controllato noto come *Walled Garden* (Giardino Murato). In questo ambiente, il produttore impone vincoli crittografici stringenti affinché l'hardware accetti esclusivamente codice firmato e autorizzato, negando all'utente finale qualsiasi privilegio di amministrazione.

1.2 Dal Modello *Black-Box* al Modello *White-Box*

La sicurezza informatica classica si fonda su assunzioni crittografiche e attacchi di tipo *Black-Box*. In tale scenario, si assume che i due attori legittimi (Alice e Bob) abbiano il controllo esclusivo dei propri dispositivi: la minaccia principale è un agente esterno che intercetta i dati in transito (attacco *Man-In-The-Middle*).

Nelle console questa assunzione crolla: ci troviamo di fronte a un modello *White-Box* in cui l'attaccante coincide con l'utente stesso. Bob possiede fisicamente la macchina e il software di Alice, ribaltando completamente il paradigma difensivo.

1.3 L'Attacco MATE (Man-At-The-End)

Questa specifica minaccia è definita nella letteratura accademica come attacco *Man-At-The-End* (MATE). In tale scenario, il dispositivo hardware e l'ambiente di esecuzione non sono considerati fidati (*Untrusted*): l'attaccante ha accesso diretto alla macchina e al software in qualsiasi condizione, a prescindere dal fatto che il software sia in esecuzione o meno. A causa di questa sproporzione di poteri, l'attaccante MATE gode di capacità che superano di gran lunga quelle del difensore. Un attaccante MATE può in particolare:

1. Tracciare ogni singola istruzione del programma in esecuzione.
2. Visualizzare e modificare il contenuto della memoria, dei registri e della cache della CPU.
3. Interrompere l'esecuzione in qualsiasi momento e analizzarla in modo *offline* (tramite *dump* di memoria).
4. Alterare il codice sorgente o i binari, e collaborare con altri attaccanti per condividere le scoperte.

1.4 La *Chain of Trust* e la *Root of Trust*

Poiché il software da solo non può difendersi in un ambiente ostile in cui l'utente può alterare la memoria a piacimento, i *Walled Garden* si affidano alla sicurezza hardware per fondare una *Trusted Computing Base*. Questa prende la forma di una *Chain of Trust* crittografica:

1. *Root of Trust (BootROM)*: Il primissimo codice eseguito all'accensione è inciso in modo immutabile nel silicio del processore. Non può essere modificato, nemmeno dal produttore.
2. Caricamento a cascata verificato: La *BootROM* verifica la firma digitale del componente successivo (il *Bootloader*) prima di passargli il controllo. Il *Bootloader* verifica il *Kernel*, e così via fino al gioco.

BootROM → Bootloader → Kernel (S.O.) → Gioco

Se in qualsiasi momento un anello della catena rileva che il codice successivo è stato alterato (*Tampering*), il processo di avvio viene interrotto. L'intero filone dell'*hacking* delle console si basa sul tentativo dell'attaccante MATE di corrompere anche uno solo di questi anelli.

2 Vulnerabilità Software: Memory Corruption

Nelle prime generazioni di console, i *Walled Garden* facevano totale affidamento su una singola barriera: la verifica della firma crittografica in ingresso. Una volta superato quel controllo, il software veniva eseguito in un ambiente privo delle moderne mitigazioni a livello di Sistema Operativo. Sfruttando i poteri di un attaccante MATE, è stato possibile compromettere la *Chain of Trust* non forzando la crittografia, ma corrompendo lo stato della memoria del programma in esecuzione.

2.1 Analisi Statica e Dinamica da parte dell'Attaccante

Per individuare vulnerabilità sfruttabili in un eseguibile chiuso, l'attaccante MATE si avvale di tecniche di ingegneria inversa. L'analisi si divide in due macro-categorie:

- **Analisi Statica:** L'attaccante analizza il codice macchina per ricostruire il *Control Flow Graph* (CFG). In assenza di adeguate tecniche di *Code Obfuscation*, il flusso di controllo risulta facilmente intelligibile tramite strumenti come IDA Pro o Ghidra.
- **Analisi Dinamica:** Eseguendo il programma in un debugger (es. GDB, OllyDbg), l'attaccante osserva la "semantica concreta" del programma, monitorando l'allocazione delle variabili in memoria e i valori dei registri della CPU in tempo reale.

2.2 Caso Studio: Stack-based Buffer Overflow (Nintendo Wii: "Twilight Hack")

Il caso di studio più emblematico di questa era è il cosiddetto "Twilight Hack" sulla Nintendo Wii. La vulnerabilità risiedeva nel gioco *The Legend of Zelda: Twilight Princess*, specificamente nel modulo di caricamento del file di salvataggio.

La meccanica dell'attacco Il programma copiava una stringa di testo (il nome del cavallo dell'avatar) dal file di salvataggio verso un buffer di dimensione fissa allocato sullo *Stack*, senza eseguire alcun controllo sui limiti (*bounds checking*).

Questa tecnica di attacco, fornire in input una stringa intenzionalmente più lunga della capienza del buffer, è stata formalizzata per la prima volta nel 1996 nell'articolo seminale di Aleph One pubblicato su *Phrack Magazine*:

"A buffer is simply a contiguous block of computer memory that holds multiple instances of the same data type. [...] To overflow is to flow, or fill over the top, brims, or bounds. We will concern ourselves only with the overflow of dynamic buffers, otherwise known as stack-based buffer overflows."

— Aleph One, *Smashing The Stack For Fun And Profit* [1], Phrack #49, 1996

I dati in eccesso invadono le locazioni di memoria adiacenti sullo *Stack* con lo scopo di sovrascrivere il *Saved Return Pointer*. Come descritto da Aleph One:

"So a buffer overflow allows us to change the return address of a function. In this way we can change the flow of execution of the program. [...] The answer is to place the code we are trying to execute in the buffer we are overflowing, and overwrite the return address so it points back into the buffer."

— Aleph One, *Smashing The Stack For Fun And Profit* [1], 1996

Alterando il *Return Pointer*, l'attaccante è riuscito a dirottare il flusso di controllo (*Control Flow Hijacking*) verso un payload malevolo (*Shellcode*) inserito all'interno del finto nome del cavallo.

2.3 Assenza di Mitigazioni: ASLR e DEP

Per inquadrare formalmente il successo di questo attacco, è utile ricorrere al framework proposto da Szekeres, Payer, Wei e Song nel paper **SoK: Eternal War in Memory** [2]¹, presentato all'IEEE Symposium on Security and Privacy 2013. Gli autori costruiscono un modello generale degli exploit di *memory corruption*:

“Memory corruption bugs in software written in low-level languages like C or C++ are one of the oldest problems in computer security. The lack of safety in these languages allows attackers to alter the program’s behavior or take full control over it by hijacking its control flow. This problem has existed for more than 30 years and a vast number of potential solutions have been proposed, yet memory corruption attacks continue to pose a serious threat.”

— Szekeres, Payer, Wei, Song, *SoK: Eternal War in Memory* [2], IEEE S&P 2013

Sulla Wii mancavano le due mitigazioni fondamentali descritte dallo studio:

- **Assenza di DEP / NX Bit** (*Data Execution Prevention*): La memoria non differenziava i permessi di esecuzione. Come spiegano gli autori, la politica $W \oplus X$ (*Write XOR Execute*) “states that a page can be either writable or executable, but not both” [2]. Sulla Wii, questa separazione non esisteva: la CPU ha letto la stringa dell’attaccante ed eseguita come codice macchina legittimo.
- **Assenza di ASLR** (*Address Space Layout Randomization*): I moduli del gioco venivano caricati agli stessi indirizzi fisici ad ogni avvio. Il paper SoK identifica l’ASLR come “the most comprehensive currently deployed protection against hijacking attacks” [2]. Questo determinismo ha permesso all’attaccante di calcolare con precisione matematica l’indirizzo da inserire nel *Return Pointer*.

Il paper chiarisce anche perché l’ASLR da sola non è sufficiente: essa “can always be bypassed if not all code and data sections are randomized” [2], evidenziando la necessità di difesa stratificata.

2.4 Il fallimento del Tamper-Proofing: Stack Canaries

L’attacco dimostra infine il limite di un software privo di tecniche di *Tamper-Proofing* attive. Il programma non implementava alcun meccanismo per calcolare l’integrità del proprio flusso di controllo, come gli *Stack Canaries*, valori sentinella inseriti tra le variabili locali e il *Return Address* per rilevare eventuali overflow prima del ritorno dalla funzione. L’alterazione malevola è passata inosservata fino all’esecuzione del payload.

2.5 Altri Casi Studio Rilevanti

Exploit “libtiff” (PlayStation Portable)

Il visualizzatore di foto integrato nella PSP analizzava file TIFF senza adeguati controlli sui limiti interni del parser. Aprendo un’immagine .tiff appositamente malformata, si innescava un *Buffer Overflow* nel parser grafico che permetteva di avviare codice non firmato, una variante diretta della vulnerabilità formalizzata da Aleph One [1], applicata a un componente di sistema invece che a un gioco.

Exploit “Soundhax” (Nintendo 3DS)

Riproducendo un file audio MP4 manipolato nel lettore musicale integrato della 3DS, si generava un *Heap Overflow*², una scrittura fuori dai limiti della memoria dinamica (*heap*), che garantiva il controllo totale del sistema. Il SoK di Szekeres et al. [2] classifica questo tipo come errore “temporale” o “spaziale” in funzione dell’oggetto corrotto nell’allocatore.

“Independence Exploit” (PlayStation 2)

Un abuso della retrocompatibilità: inserendo un file di salvataggio (TITLE.DB) con un nome eccessivamente lungo all’interno di una Memory Card della PS1, si causava un *Buffer Overflow* letale nella routine di parsing della più moderna PS2.

¹Titolo completo: *SoK: Eternal War in Memory*, presentato all’IEEE Symposium on Security and Privacy 2013.

²Una scrittura fuori dai limiti della memoria dinamica (*heap*).

3 Protezione della copia e crittografia White-Box

3.1 Contesto storico e tecniche di attacco

Il contesto storico è quello dei primi anni 2000 – periodo in cui erano appena entrate in commercio console come Xbox, GameCube e PlayStation 2.

Come anticipato in introduzione, il modello *MATE* assume in partenza che tutto ciò che si trova all'esterno del processore (RAM, bus di sistema, periferiche, ...) sia esposto e potenzialmente sotto il controllo dell'avversario.

Il principio è lo stesso che giustifica la crittografia White-Box: chi esegue il programma è al contempo chi può attaccarlo.

Il contributo di *Weidong Shi, Hsien-Hsin S. Lee, Chenghuai Lu e Tao Zhang*, pur non essendo specifico dell'ambito videoludico, esamina **Attacchi e analisi del rischio per i sistemi hardware a supporto della protezione dalla copia del software** [3]³ e cita tra le principali tecniche di attacco hardware per soverchiare le difese software:

- **Trace Analysis:** l'attaccante raccoglie informazioni del software protetto tramite *logging* ed analizzando le tracce d'esecuzione dello stesso. Alcune primitive sono riconoscibili anche dopo essere state cifrate (ad esempio branch condizionali, indici e diversi tipi di variabili).
- **Mod-chip:** con il termine *mod-chip* gli autori si riferiscono a dispositivi designati a mettere alla luce i meccanismi di protezione di copia. Possono anche essere destinati all'appropriazione del segnale di un altro dispositivo o a lanciare attacchi di spoofing ad altri dispositivi.
- **Emulatori:** è complesso combattere contro attacchi di emulazione senza utilizzare la cifratura; tuttavia, non è possibile parlare di protezione della copia del software se il suddetto può essere eseguito su un emulatore senza autorizzazione.

Scendendo ulteriormente nel dettaglio, gli autori analizzano anche altri contesti ed alcuni dei relativi attacchi:

- Analizzando le **lazy authentication**⁴ e le **counter-mode encryption**⁵, si può parlare ad esempio di **attacco ATT**, ovvero *"alter then trace attack"*, il quale prevede che l'attaccante sia in grado di modificare una parte del software (programma o dati) bit-per-bit, che le istruzioni alterate possano essere eseguite e che l'integrità di tali istruzioni/dati non venga verificata tempestivamente né in modo rigoroso istruzione per istruzione.
Il rischio delle autenticazioni "Lazy" può essere ridotto utilizzando *control flow/memory fetch address obfuscation* hardware, nonostante questo possa aumentare notevolmente l'overhead e la complessità dell'hardware stesso.
- Riguardo il **Software slice**, una tecnica che prevede di mettere in sicurezza il software proteggendone solo alcune porzioni, l'attaccante può trattare le porzioni di codice protetto come *black box* e cercare di creare un codice diverso dall'originale ma che riesca ad emettere lo stesso output dallo stesso input – naturalmente si tratta di un problema computazionale complesso se applicato a qualsiasi programma, ma l'avversario potrebbe essere interessato nel corrompere anche solo una *slice* di programma, e potrebbe riuscire a farlo in un tempo ragionevole.
- Gli autori del paper analizzano poi gli **Active Information Flow Attack**. Il contesto prevede che alcuni sistemi utilizzino lo stesso schema di cifratura e la stessa chiave sia per proteggere le istruzioni che i dati dinamici, esponendo il controllo della protezione all'utente tramite istruzioni come **enter security**, **exit security**, **secure load** e **secure store**⁶.
Vengono portati due esempi di attacco a questi sistemi: nel primo, l'attaccante può semplicemente rimpiazzare parti di codice non protetti con contenuto malevolo; dopo che il codice infetto viene criptato, l'attaccante può ridirezionare il puntatore verso tale codice per farlo eseguire in un contesto

³Titolo originale: *Attacks and Risk Analysis for Hardware Supported Software Copy Protection Systems*

⁴Con 'lazy' ci si riferisce alla situazione nella quale la sorgente dei dati è utilizzata come un operando ed il risultato della computazione ottenuto utilizzando dati non autenticati è in grado di modificare lo stato del processore prima che l'integrità di tali dati sia confermata.

⁵La counter-mode encryption (CTR), invece di cifrare il messaggio direttamente, si cifra una sequenza di valori numerici incrementali per produrre un flusso di bit pseudo-casuali, detto pad. Il messaggio viene poi combinato con questo pad tramite XOR, ottenendo il ciphertext. Fonte: *Comments to NIST concerning AES Modes of Operations: CTR-Mode Encryption* [4], Lipmaa et al.

⁶Fonte: Table 3, *Attacks and Risk Analysis for Hardware Supported Software Copy Protection Systems* [3], Shi et al.

protetto. Nel secondo esempio, l'attaccante cerca di sfruttare le *linked list* per ridirezionare l'ultimo puntatore della lista ad un elemento segreto per poterlo rivelare in uno spazio non protetto. Le soluzioni a questo tipo di attacchi prevedono anche l'impiego di *program analysis* o di diverse tecniche di compilazione.

- In alcuni contesti potrebbe essere necessario utilizzare **cifrature semplici o chiavi corte**, per motivi di performance o di limitazioni hardware. Anche in questo contesto, gli autori presentano due semplici esempi di attacco, uno che mira alla derivazione di un *integrity code* breve a protezione di una porzione di codice tramite *brute force attack*, mentre l'altro che sfrutta le caratteristiche cicliche delle variabili contatore, come il fatto che incrementano in modo regolare, per individuarle (anche se cifrate – ma con una chiave corta) e ricavarne le locazioni in memoria.

3.2 Case Study: la sicurezza di Xbox (OG)

Il seguente case study è basato sull'analisi di *Michael Steil* nel paper **17 Mistakes Microsoft Made in the Xbox Security System** [5].

L'originale Xbox, introdotta sul mercato da Microsoft nel 2001, era stata ideata per essere utilizzata senza software copiati, applicazioni non ufficiali e sistemi operativi alternativi; tutti i sistemi di sicurezza, pertanto, erano stati implementati per adempiere a questo scopo.

In termini di hardware, per ridurre le tempistiche di sviluppo e costruzione, Microsoft ha optato per creare la sua console a partire da componenti per PC *off the shelf*, mentre per il software si è affidata alle tecnologie proprietarie di Windows e DirectX come basi della console.

Nonostante gli errori compiuti da Microsoft in questa console siano diversi, vogliamo concentrarci solo sulla concretizzazione di quanto analizzato fino ad ora in questo capitolo.

3.2.1 Trace Analysis e *bus sniffing*

L'autore cita il lavoro di Andrew "bunnie" Huang⁷, al tempo studente PhD al MIT, il quale una volta disassemblata la console ha ricavato moltissime informazioni dalla flash memory: analizzandone il contenuto, trovò nei 512 byte superiori il codice di una vecchia versione della ROM rimasta per errore nel build finale – uno spazio che invece sarebbe dovuto rimanere libero, dedicato ad una porzione di memoria denominata d'ora in avanti in *secret ROM*, destinata al setup della console e dal codice di decriptazione della memoria flash. Bunnie scoprì che questo codice era un interpreter per le tabelle in memoria flash, ed in aggiunta una funzione di decriptazione che somigliava ad *RC4*⁸.

Dal momento che, indipendentemente da quali operazioni compiesse su questa porzione di memoria, la console si avviava correttamente, capì che doveva necessariamente essere sovrascritta ad ogni avvio, e che quello era il luogo dove si trovava la *secret ROM*.

Sfruttando le risorse del laboratorio hardware del MIT, Huang costruì un hardware dedicato per 'sniffare' il bus *HyperTransport*⁹, ritenuto inaccessibile da Microsoft, estraendo così la *secret ROM* completa e la chiave *RC4*.

3.2.2 Mod-chip

Dopo aver ottenuto le chiavi di crittografia, comparvero i mod-chip: alcuni rimpiazzavano interamente la memoria flash, mentre altri ne 'patchavano' solo una parte, ed utilizzavano per la gran parte il chip originale.

Se l'Xbox veniva avviata senza alcuna flash memory installata, essa tentava di leggere dal bus LPC; questa scoperta ha portato allo sviluppo di chip che inducevano la console a 'credere' di non avere alcuna memoria installata, ed il *fallback* era un chip custom che permetteva di copiare interi giochi sull'hard disk della console e di lanciarli da lì.

3.2.3 Emulatori e software non autorizzato

L'idea originale di Microsoft era quella di escludere qualsiasi software che non provenisse dal supporto originale o che non fosse autorizzato da Microsoft. Il design prevedeva che il sistema di sicurezza rendesse

⁷Bunnie's adventures hacking the Xbox

⁸RC4: Uno tra i più famosi e diffusi algoritmi di cifratura a flusso a chiave simmetrica, utilizzato ampiamente in protocolli quali l'SSL ed il WEP.

⁹HyperTransport (HT): Conosciuto anche come *Lightning Data Transport*, è una tecnologia destinata all'interconnessione tra processori di un computer.

impossibile installare Linux sulla console, come anche una qualsiasi versione di software senza licenza. Da un lato, l'idea rendeva il sistema di sicurezza più semplice e riduceva i punti di attacco, ma dall'altro ha triplicato i potenziali attaccanti: nonostante la comunità Open Source, gli sviluppatori *homebrew* e i *cracker* avessero interessi molto diversi tra loro, in questo caso potevano unirsi e attaccare congiuntamente il sistema di sicurezza Xbox.

3.2.4 Lazy authentication e verifica non rigorosa

Il sistema di sicurezza Xbox prevedeva che la *secret ROM* verificasse l'autenticità del secondo bootloader (*2bl*) prima di eseguirlo: a questo scopo, Microsoft scelse di utilizzare *RC4* sia come algoritmo di decrittazione che come funzione di hash, confrontando gli ultimi 32 bit decifrati con una costante nota. Tuttavia, *RC4* è un cifrario a flusso che decifra ogni byte in modo indipendente: modificare un byte del testo cifrato non compromette la decifratura dei byte successivi. La verifica degli ultimi 32 bit risultava quindi priva di qualsiasi valore – non esisteva alcun hash reale. Il *2bl* poteva essere modificato arbitrariamente senza che il sistema se ne accorgesse.

3.2.5 Cifrature semplici e chiavi corte

I vincoli di spazio della *secret ROM* – grande solo 512 byte – hanno costretto Microsoft a scegliere algoritmi crittografici estremamente compatti. La scelta ricadde su *RC4* con una chiave a 16 byte, archiviata nella stessa *secret ROM*.

Una chiave così corta, una volta che il bus *HyperTransport* fu sniffato da Huang, risultò immediatamente estraibile. Nella "versione 1.1"¹⁰ Microsoft aggiunse *TEA* (*Tiny Encryption Algorithm*) come hash, scelto verosimilmente per le sue dimensioni ridotte; tuttavia, come documentato da Schneier in *Applied Cryptography*, *TEA* non è sicuro se utilizzato come funzione di hash – invertendo il 16-esimo ed il 31-esimo bit di una parola a 32 bit si ottiene lo stesso valore di hash, rendendo semplicissima la manipolazione del *2bl*.

4 Attacchi Hardware: Fault Injection e Glitching

Con la maturazione delle tecniche di Secure Coding e la corretta implementazione della crittografia, le architetture dei *Walled Garden* sono diventate logicamente inattaccabili a livello software. Tuttavia, in uno scenario *MATE* l'attaccante possiede il dispositivo fisico. Quando il software non presenta vulnerabilità sfruttabili, il nuovo vettore di attacco diventa l'hardware stesso, invalidando l'assunzione fondamentale dell'informatica: che la CPU esegua le istruzioni in modo deterministico e corretto.

4.1 Il Limite dell'Environment Checking

Le tecniche di *Tamper-Proofing* più avanzate prevedono l'*Environment Checking*: un programma blindato verifica non solo la propria integrità, ma anche l'autenticità dell'ambiente (il processore e la memoria) in cui gira.

Tuttavia, il software è un'astrazione che dipende totalmente dall'hardware sottostante. Come evidenziato da Barenghi, Breveglieri, Koren e Naccache nel paper **Fault Injection Attacks on Cryptographic Devices** [6]¹¹:

“Although the current standard cryptographic algorithms proved to withstand exhaustive attacks, their hardware and software implementations have exhibited vulnerabilities to side channel attacks, e.g., power analysis and fault injection attacks. This paper focuses on fault injection attacks that have been shown to require inexpensive equipment and a short amount of time.”

— Barenghi, Breveglieri, Koren, Naccache, *Fault Injection Attacks on Cryptographic Devices* [6], Proceedings of the IEEE, 2012

Se l'attaccante riesce ad alterare fisicamente le condizioni operative del chip (voltaggio, frequenza di clock, temperatura), può indurre la CPU a commettere errori di calcolo infinitesimali. In questo scenario, la funzione software di verifica fallisce nel rilevare l'anomalia perché è la CPU stessa a "mentire" al software.

¹⁰Chiamata così dalla community di hacker, si trattava di un update della ROM in seguito alla scoperta di Bunnie.

¹¹Titolo originale: *Fault Injection Attacks on Cryptographic Devices: Theory, Practice, and Countermeasures*, pubblicato su Proceedings of the IEEE, 2012.

4.2 La Tecnica: Fault Injection (Voltage Glitching)

La manipolazione fisica del processore per indurre errori prende il nome di *Fault Injection*. Timmers e Spruyt (2016) nella presentazione **Bypassing Secure Boot using Fault Injection** [7]¹² forniscono la definizione operativa:

“Introducing faults in a target to alter its intended behavior.” [...]
 `if( key_is_correct ) { open_door(); } else { keep_door_closed(); }, Glitch here!`
— Timmers, Spruyt, *Bypassing Secure Boot using Fault Injection* [7], Black Hat EU 2016

Gli autori classificano le tecniche di iniezione in quattro categorie principali: manipolazione del clock, del voltaggio, di campi elettromagnetici e del laser. La variante più accessibile e utilizzata nell’hacking delle console è il *Voltage Glitching*.

La procedura tecnica è la seguente:

1. Un chip programmabile (FPGA o microcontrollore) viene saldato ai pin di alimentazione della CPU della console.
2. Il dispositivo monitora i segnali elettrici e attende il ciclo di clock esatto in cui la CPU sta eseguendo l’istruzione critica (es. la comparazione della firma digitale).
3. In quell’istante, il chip abbassa repentinamente la tensione per una frazione di microsecondo (il *Glitch*).
4. La CPU, “stordita” dal calo di energia, non si spegne ma non riesce a completare l’istruzione correttamente: il risultato tipico è l’*Instruction Skip* (salto dell’istruzione di controllo) o la corruzione di un registro.

Come notano Timmers e Spruyt, la conseguenza pratica è che “*the true fault model is hard to predict or prove*” [7]: i fault introdotti possono corrompere registri, bus di memoria o il flusso di istruzioni stesse, con effetti che variano da chip a chip.

4.3 Caso Studio 1: Reset Glitch Hack (Xbox 360)

L’architettura di sicurezza della Xbox 360 era considerata un capolavoro ingegneristico basato su un *Secure Boot* crittograficamente solido. Non potendo decifrare il codice, gli hacker hanno sviluppato il *Reset Glitch Hack* (RGH).

L’attacco prendeva di mira la *Chain of Trust* al momento dell’accensione. Un *Modchip* saldato sulla scheda madre inviava un impulso elettrico al pin di “Reset” della CPU nell’esatto momento in cui il *Bootloader* stava verificando l’hash crittografico (tramite la funzione `memcmp`) del Kernel. Il glitch forzava la funzione a restituire un risultato positivo (firma valida) anche per un Kernel modificato, permettendo l’esecuzione di un sistema operativo alterato e sbloccando totalmente la console.

4.4 Caso Studio 2: Voltage Glitching del BootROM (Nintendo Switch, versioni “Patchate”)

Mentre i primi modelli di Nintendo Switch presentavano una vulnerabilità puramente software nel BootROM (il *Fusée Gelée*, un *Buffer Overflow* nella gestione del protocollo USB), Nintendo corresse il difetto nelle revisioni hardware successive (versioni “patchate”, Switch Lite e OLED).

Poiché il BootROM è inciso fisicamente nel silicio e la crittografia era stata corretta, gli attaccanti sono ricorsi nuovamente alla *Fault Injection*. I moderni Modchip per Nintendo Switch (come HWFLY o Picofly) utilizzano il *Voltage Glitching* per corrompere l’esecuzione della CPU Tegra X1 esattamente durante la prima fase di BootROM, saltando il controllo della firma del Bootloader. Questo scenario è il caso concreto della dimostrazione di Timmers e Spruyt [7]: quando la superficie logica è ridotta e prova bug è improbabile, “*other attack(s) must be used when not logically flawed*”.

¹²Presentata alla conferenza Black Hat Europe 2016.

4.5 Caso Studio 3: Xbox One. Voltage Glitching del Secure Boot

Anche la Microsoft Xbox One è stata compromessa tramite voltage glitching applicato al processore di sicurezza ausiliario (il *Platform Security Processor*). Gli attaccanti hanno iniettato un guasto durante la verifica della firma del firmware, ottenendo l'esecuzione di codice non autorizzato in un contesto con privilegi elevati, dimostrando che la tecnica del Glitching trasferiva le medesime dinamiche dalla generazione precedente a quella successiva.

4.6 Implicazioni e Contromisure

Gli attacchi di *Fault Injection* dimostrano un paradigma cruciale: la sicurezza del software non può essere garantita se l'hardware non è resiliente alle alterazioni fisiche. Barengi et al. [6] classificano le contromisure in due famiglie principali:

1. **Ridondanza spaziale e temporale:** eseguire lo stesso controllo crittografico più volte in momenti diversi, confrontando i risultati. Un singolo glitch corromperà uno solo dei risultati, rendendolo rilevabile.
2. **Sensori fisici** (*Voltage Detectors*): circuiti dedicati che monitorano continuamente la tensione di alimentazione e spengono immediatamente il sistema al rilevamento di anomalie.

5 Architetture Moderne

Riguardo l'analisi delle architetture moderne, ovvero quelle che caratterizzano le console degli anni 2020, la letteratura non offre molti approfondimenti.

I riferimenti di questa sezione provengono dal documento “*Next-Gen Exploitation: Exploring the PS5 Security Landscape*” [8] redatto da SpecterDev¹³, un ricercatore di sicurezza specializzato in kernel exploitation e virtualizzazione, con focus su Android e Linux e che si occupa di sicurezza delle console come attività secondaria dal 2020, con particolare attenzione all'hypervisor di PlayStation 5.

Abbiamo estrapolato dal documento solo alcune delle tecniche difensive adottate da questa nuova console, esposte nei seguenti paragrafi.

XOM e kCFI come ostacolo all'analisi del codice Il kernel implementa anche il software *Control Flow Integrity* (da cui kCFI), che ostacola l'analisi del *Control Flow Graph*.

Al contempo, l'utilizzo della XOM – *eXecute-Only Memory* – mitiga la *Return Oriented Programming* rendendo più complesso trovare i **gadget**, piccoli pezzi di codice che terminano ognuno con un'istruzione **ret** e che, se concatenati, potrebbero portare alla costruzione di operazioni arbitrarie senza mai scrivere nuovo codice. La XOM ostacola sia l'analisi statica (il codice non può essere letto) sia quella dinamica (non si può copiare il binario a *runtime*). I gadget richiedono quindi un attacco **brute force** per essere recuperati.

La XOM, inoltre, non può essere disabilitata con *kernel R/W* arbitrarie poiché le *Nested Page Tables* – che mappano gli indirizzi virtuali a quelli fisici della RAM – non sono accessibili direttamente; la zona proibita è definita dall'**hypervisor** e non dal kernel.

L'hypervisor AMD SVM come environment checker L'hypervisor di PlayStation 5 è un monitor di sicurezza che intercetta le azioni sensibili del kernel guest. Sfrutta l'AMD SVM (*AMD Secure Virtual Machine/Virtualization*), la tecnologia AMD per la virtualizzazione dell'hardware-backend, per dare vita ad una vera e propria *Virtualization-Based security* per le console.

Ogni volta che il kernel tenta di eseguire una azione sensibile, l'hardware genera un **VMEXIT**: interrompe l'esecuzione e passa il controllo all'hypervisor, il quale decide se l'operazione è lecita. Si assicura che il kernel non possa disabilitare componenti di sicurezza essenziali (ad esempio modificando i bit relativi a *Write Protect* [WP], *Supervisor Mode Access Protection* [SMAP] o *Supervisor Mode Execution Protection* [SMEP]), oppure modificando i *Model-Specific Registers* [MSR].

¹³Dayzerosec — About SpecterDev

Isolamento crittografico nel co-processore sicuro Su PlayStation 5 le applicazioni e le librerie sono **cifrate su disco**. I file si chiamano SELF e solo i co-processori sicuri (MP0 di Sony e MP3 di AMD PSP) ne possiedono le chiavi. Il kernel x86 fa da intermediario per le richieste di decifratura e verifica, ma non ha mai accesso diretto alle chiavi.

Una grande differenza tra PlayStation 4 e PlayStation 5 è che sulla prima questa API del kernel poteva essere patchata, mentre sulla seconda l'hypervisor lo impedisce.

PS4 vs PS5: evoluzione del modello di post-exploitation In questa ultima sezione, abbiamo voluto evidenziare con un confronto diretto le maggiori differenze a livello di sicurezza tra PlayStation 5 e la sua antecedente PlayStation 4.

Sulla vecchia PlayStation 4 le mitigazioni erano deboli, e spesso una *kernel R/W* era sufficiente – l'assenza di SMAP permetteva di costruire oggetti kernel falsi direttamente nello spazio utente e non c'era nemmeno bisogno di avere a che fare con la *kernel Address Space Layout Randomization* [kASLR], che mescola randomicamente la posizione in memoria del kernel ad ogni avvio, per non rendere noto il luogo in memoria fisso dove si trovano le strutture sensibili.

Su PlayStation 5 gli unici attacchi possibili sono i *data-only attacks*¹⁴, e l'unico modo per accedere senza bug nell'hypervisor è tramite lo spoofing della mailbox PSP (che tuttavia l'autore non ha verificato).

6 Conclusioni e considerazioni

Con questo elaborato abbiamo voluto approfondire lo stato dell'arte dei sistemi videoludici dall'inizio della loro storia fino ai nostri giorni.

La conclusione che abbiamo raggiunto è che, all'interno di un *Walled Garden*, la protezione non è una condizione statica, bensì una continua ricerca sia da parte dei difensori che degli attaccanti.

Nelle prime generazioni, l'assenza di mitigazioni basilari ha reso i sistemi vulnerabili a dirette alterazioni della "semantica concreta" del programma tramite banali corruzioni della memoria (*Buffer Overflow*).

In seguito, abbiamo notato come l'applicazione teorica della crittografia fallisce se l'implementazione software o l'infrastruttura hardware non sono esplicitamente progettate per resistere a un contesto operativo *White-Box* ostile.

Oggi, il paradigma difensivo è profondamente mutato. Accettando il compromesso che sia impossibile scrivere codice totalmente esente da bug o produrre hardware fisicamente inattaccabile, l'ingegneria della sicurezza ha abbracciato gli approcci *Defense-in-Depth* e *Least Privilege*.

Le architetture moderne non si limitano più a tentare di bloccare l'attaccante al perimetro del *Secure Boot*, ma lo confinano attivamente isolando i processi (*Sandboxing*) e virtualizzando l'hardware (*Hypervisor*). Di conseguenza, la compromissione non è più affidata a un singolo bug risolutivo, ma richiede l'elaborazione di complesse *Exploit Chain*, che combinano tecniche avanzate come il ROP per aggirare molteplici strati di mitigazione consecutivi.

In conclusione, lo studio della sicurezza nelle console dimostra che l'obiettivo finale del *Tamper-Proofing* e della protezione del software in scenari *MATE* non è l'invulnerabilità assoluta, un traguardo teoricamente irraggiungibile quando l'attaccante possiede fisicamente il dispositivo. Lo scopo è, piuttosto, innalzare la complessità tecnica, il tempo e il costo dell'attacco fino a renderlo impraticabile per la stragrande maggioranza degli attori, garantendo così l'integrità e la sopravvivenza economica dell'ecosistema chiuso.

¹⁴ Attacchi che non modificano il flusso di controllo del programma: l'attaccante sfrutta un bug di scrittura in memoria per corrompere dati, non codice [9]

7 Fonti

1. *Smashing The Stack For Fun And Profit*, Aleph One, Phrack #49, 1996
2. *SoK: Eternal War in Memory*, Szekeres, Payer, Wei, Song, IEEE S&P 2013
3. *Attacks and Risk Analysis for Hardware Supported Software Copy Protection Systems*, Shi et al.
4. *Comments to NIST concerning AES Modes of Operations: CTR-Mode Encryption*, Lipmaa et al.
5. *17 Mistakes Microsoft Made in the Xbox Security System*, Michael Steil, Xbox Linux Project
6. *Fault Injection Attacks on Cryptographic Devices: Theory, Practice, and Countermeasures*, Barenghi, Breveglieri, Koren, Naccache, Proceedings of the IEEE, 2012
7. *Bypassing Secure Boot using Fault Injection*, Timmers, Spruyt, Black Hat EU 2016
8. *Next-Gen Exploitation: Exploring the PS5 Security Landscape*, SpecterDev
9. *Data-Only Attacks Are Easier than You Think*, Johannesmeyer et al.